

Kubernetes



Наставник: Казьмин Роман

Дата: 08.04.2026

Кому будет полезно?

- **Разработчикам**

Запуск приложений в идентичной среде (на ноутбуке и на проде) без воплей «а у меня на компьютере всё работало!».

- **DevOps и Системным инженерам:**

Автоматизация деплоя, самовосстановление системы (Self-healing) и управление инфраструктурой как кодом.

- **QA-инженерам:**

Возможность мгновенно разворачивать полные копии окружения для тестирования новых фич.

- **Менеджерам и Бизнесу:**

Сокращение Time-to-Market и эффективное использование серверных ресурсов (экономия денег).



План активности

Часть 1

- Вступление
- Пролог: жизнь до Kubernetes
- Модуль 1: Pod, Deployment, Liveness Probe
- Модуль 2: Service и DNS
- Модуль 3: ConfigMap и Secret

Часть 2

- Модуль 4: Ingress
- Модуль 5: Масштабирование
- Модуль 6: RollingUpdate и Rollback
- Финал: общая схема кластера



Kubernetes для джуниоров

StreamFlow

сервис транскодирования

Kubernetes огромный.

За 1.5 часа не охватить всё — и незачем.

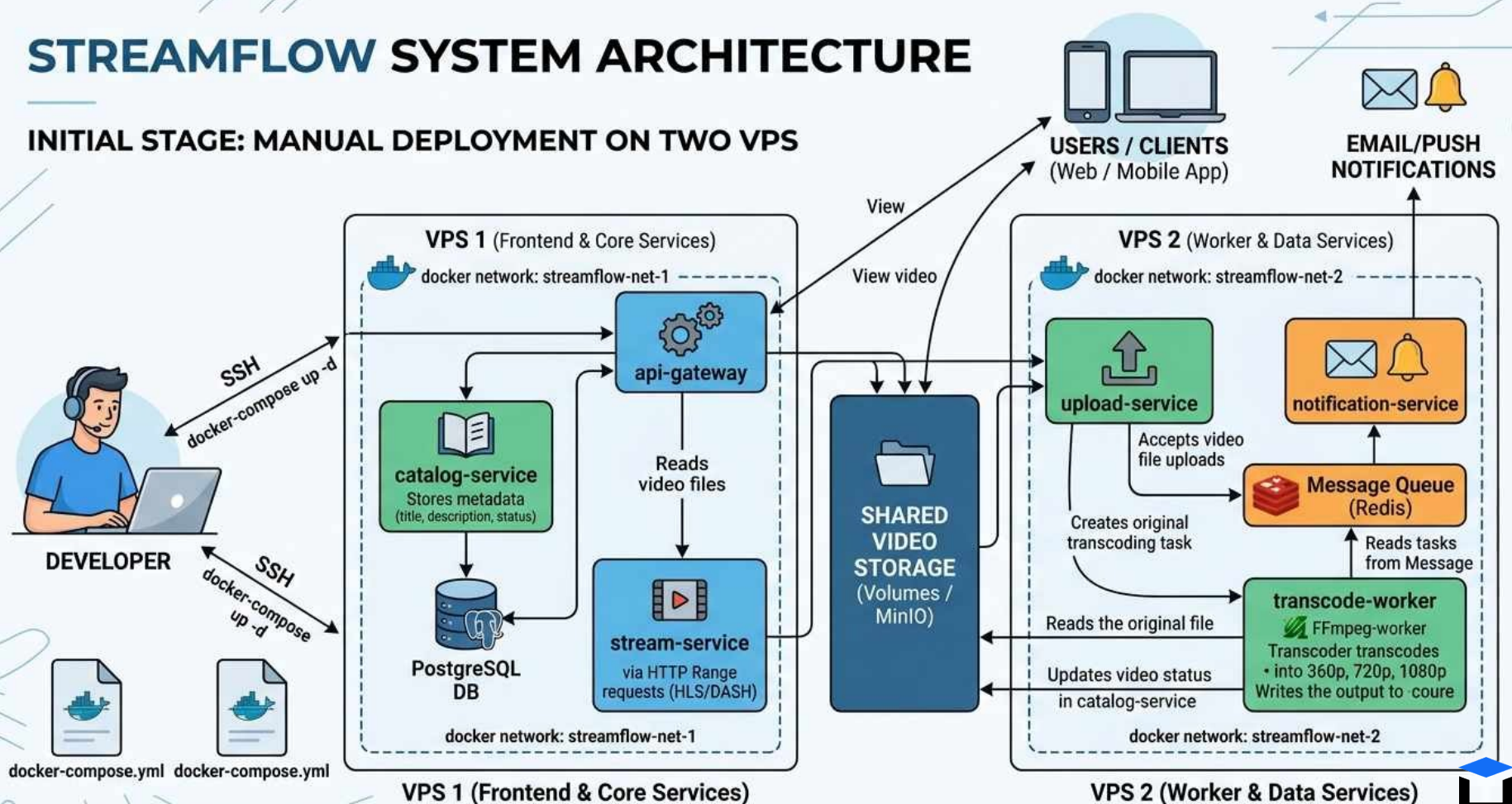
Мы возьмём 6 концепций которые встречаются в любом реальном проекте.

После этого занятия у вас будет фундамент для самостоятельного движения вперёд.



STREAMFLOW SYSTEM ARCHITECTURE

INITIAL STAGE: MANUAL DEPLOYMENT ON TWO VPS



ЖИЗНЬ ДО K8S — КАК ЭТО ВЫГЛЯДИТ

Слайд 1

БОЛЬ:



Деплой = ssh + docker pull
+ docker-compose up -d.
Вручную. Каждый раз.

Типичный деплой-скрипт
команды:

```
ssh user@185.12.44.10
cd /opt/streamflow
git pull
docker-compose pull
docker-compose up -d --
-no-deps transcode-worker

# Молимся, что ничего
не сломалось 🙏
```



Нет истории
кто, что и когда
деплоил?



Нет rollback
только "git stash +
снова вручную"



Разные версии
на серверах
"у меня работает"



Ночные деплои
потому что страшно
делать днём

ИТОГ:

Сформулировать
запрос:



Хотим **reproducible**
(Воспроизводимые)
deploys

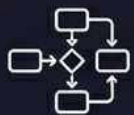


Хотим **auditable**
(Проверяемые)
deploys

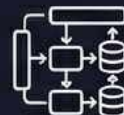


Хотим **safe**
(Безопасные)
deploys





ДЕКЛАРАТИВНЫЙ ПОДХОД



● Императивно (как раньше)

```
ssh user@185.12.44.10

docker stop transcode-worker

docker pull streamflow:v2

docker run -d --name transcode-worker \
  --restart=always streamflow:v2

# И молимся...
```

Говорим **КАК** сделать. Шаг за шагом. Вручную.

● Декларативно (Kubernetes)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: transcode-worker
spec:
  replicas: 2
  template:
    spec:
      containers:
        - image: streamflow:v2
```

Говорим **ЧТО** хотим. Kubernetes разберётся как.

✓ Один файл = весь деплой. Можно хранить в git. Можно проверять. Можно воспроизвести



1

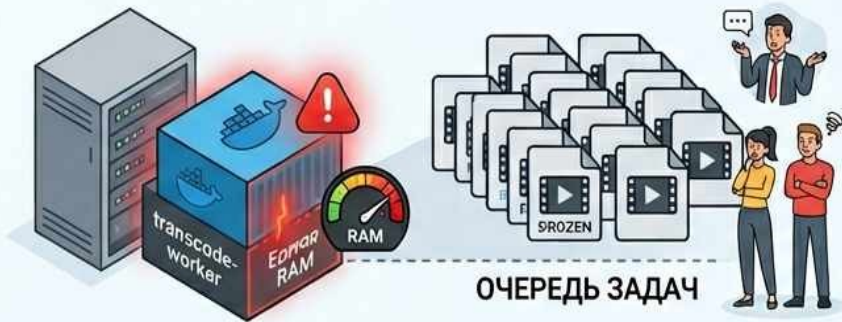
Как запустить и не потерять

Pod · Deployment · Liveness Probe



Слайд 2 · Что происходит когда падает transcode-worker

БОЛЬ: Воркер завис на тяжёлом файле. Очередь встала. Никто не знает



ПОСЛЕДОВАТЕЛЬНОСТЬ СОБЫТИЙ



РЕШЕНИЕ: k8s + liveness probe: k8s сам обнаружит зависший контейнер и перезапустит его. Без будильника в 8 утра

✗ Раньше

Обнаружение: случайно/утром

Реакция: человек + ssh

Время простоя: часы

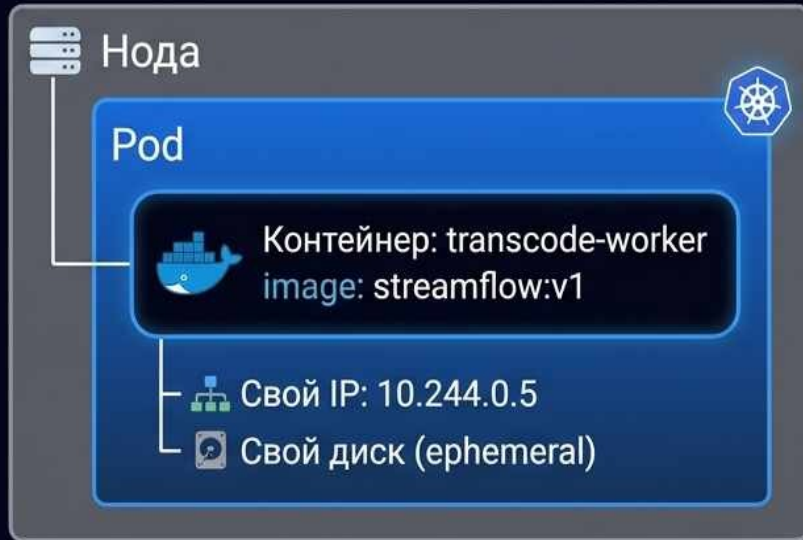
✓ Теперь

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  failureThreshold: 3
```

Реакция: авто, секунды

POD — БАЗОВАЯ ЕДИНИЦА KUBERNETES

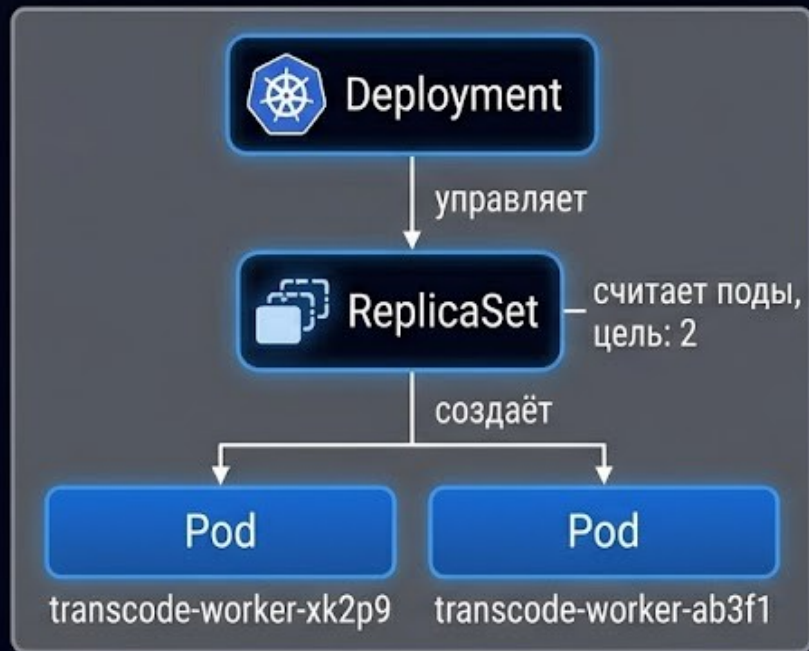
Deployment упал → нам нужно понять из чего он состоит



```
apiVersion: v1
kind: Pod      ← тип объекта
metadata:
  name: transcode-worker-xk2p9  ← имя (случайное!)
spec:
  containers:
  - name: worker
    image: streamflow:v1  ← используемый Docker-образ
    ports:
    - containerPort: 8080
```

! Pod одноразовый: упал → удалён навсегда | IP потерян | Kubernetes не воскресит
→ Нужен кто-то кто следит за подами. Это Deployment →

DEPLOYMENT — КТО СЛЕДИТ ЗА ПОДАМИ



✓ Вы ничего не делаете. Deployment сам поддерживает нужное количество подов. Поддерживает нужно е количество подов.

LIVENESS PROBE — КАК ЭТО РАБОТАЕТ



livenessProbe:

httpGet:

path: /healthz

port: 8080

initialDelaySeconds: 3

periodSeconds: 5

failureThreshold: 3

← эндпоинт который мы проверяем

← подождать перед первой проверкой

← проверять каждые 5 секунд

← сколько провалов до перезапуска





СОСТОЯНИЯ ПОДА – ЧТО ВИДИМ В ТЕРМИНАЛЕ



CrashLoopBackOff:

```
kubectl logs <под> --previous
```

смотрим логи до падения

ImagePullBackOff:

```
kubectl describe pod <под>
```

смотрим Events

Running но не работает:

```
kubectl exec -it <под> -- sh
```

заходим внутрь

⚠ **kubectl get pods** – первое что открываешь когда что-то сломалось



Слайд 3 · Первый деплой в minikube — от docker-compose к Deployment

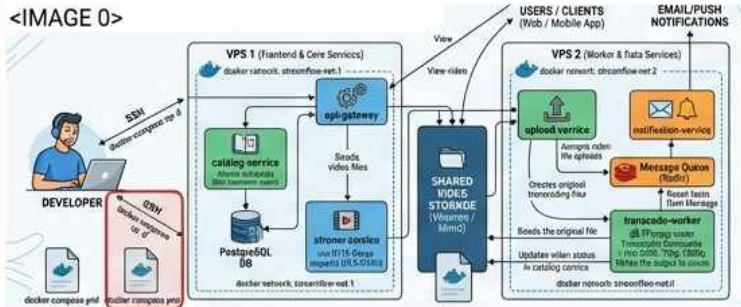
✗ Раньше

✓ РЕШЕНИЕ: Берём transcode-worker и описываем его как Deployment. Это наш первый манифест.

```
# docker-compose.yml
transcode-worker:
  image: streamflow/transcode:1.2
  environment:
    - REDIS_URL=redis:6379
  restart: always
```



<IMAGE 0>

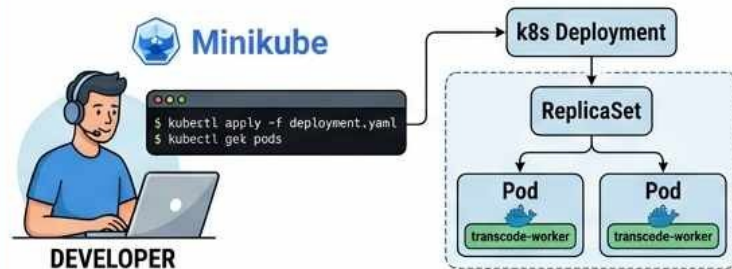


docker-
compose.yml
→
k8s
Deployment:

✓ Теперь

k8s Deployment

```
# deployment.yaml
kind: Deployment
spec:
  replicas: 2
  template:
    spec:
      containers:
        - image: streamflow/transcode:1.2
```



Что получили сразу бесплатно:



restart: always → k8s
делает это сам через
контроллер



Две реплики — одной
командой (replicas: 2)



История деплоев —
kubectl rollout history



Rollback —
kubectl rollout undo



Демо · Модуль 1 · Pod и Deployment

Запускаем transcode-worker, убиваем под, наблюдаем как Kubernetes его поднимает

```
# 1. Направляем Docker в контекст Minikube
eval $(minikube docker-env)

# 2. Применяем манифест
kubectl apply -f deployment.yaml

# 3. Смотрим что поднялось
kubectl get pods
kubectl get all

# 4. Убиваем под вручную — Deployment поднимет новый
kubectl delete pod <pod_name>
kubectl get pods -w

# 5. Смотрим события пода (liveness probe)
kubectl describe pod <pod_name>
# Найдите секцию Events — там история проверок
```



Ожидаемый результат: после `kubectl delete pod` новый под появляется за ~5 секунд

Демо · Модуль 1 · Отладка состояний

Симулируем ошибки и учимся их читать

```
# ImagePullBackOff – меняем образ на несуществующий
# image: streamflow-mock:v999
kubectl apply -f streamflow.yaml
kubectl get pods
# Видим: ImagePullBackOff
kubectl describe pod <pod_name>
# Events → Failed to pull image

# CrashLoopBackOff – смотрим логи до падения
kubectl logs <pod_name> --previous

# Возвращаем правильный образ
# image: streamflow-mock:v1
kubectl apply -f streamflow.yaml
kubectl get pods -w
# Статусы: Pending → ContainerCreating → Running
```



2

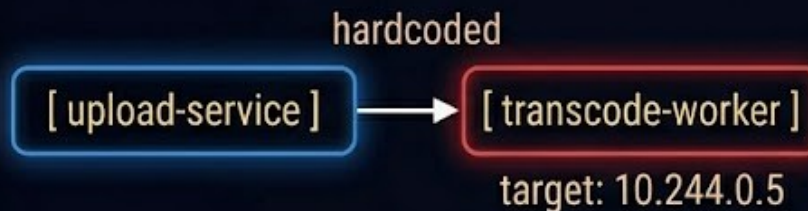
Сервисы находят друг друга

Service · DNS



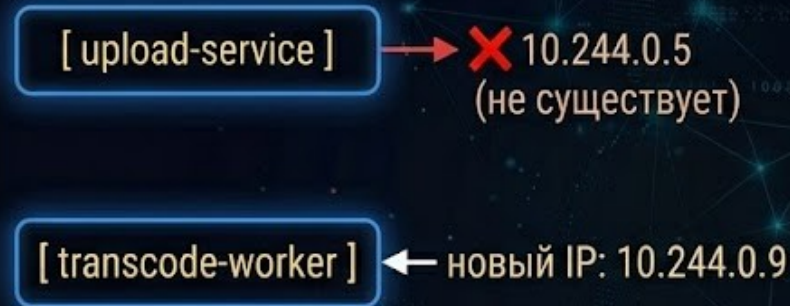
ПРОБЛЕМА – СЕРВИСЫ НЕ НАХОДЯТ ДРУГ ДРУГА

До перезапуска



```
# в коде upload-service  
TRANSCODE_URL = "http://10.244.0.5:8080"  
# Работает... пока под не перезапустится
```

После перезапуска пода

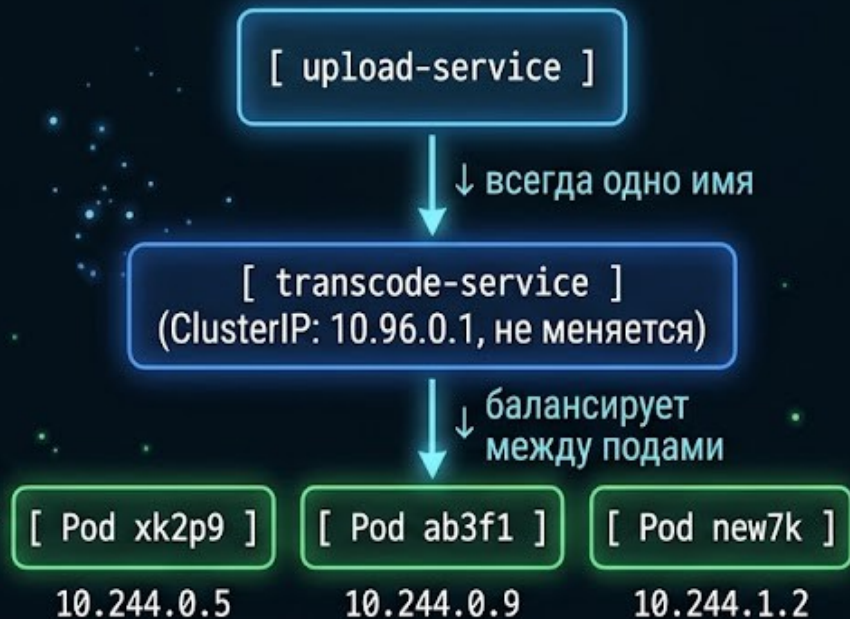


Под получил новый IP. upload-service не знает.
500 Internal Server Error.

❌ IP подов меняется при каждом перезапуске.
Хардкодить IP в Kubernetes – всегда ошибка.



SERVICE — СТАБИЛЬНОЕ ИМЯ ВМЕСТО IP



Поды приходят и уходят. Service — постоянный.

```
apiVersion: v1
kind: Service
metadata:
  name: transcode-service ← DNS-имя внутри кластера
spec:
  selector:
    app: transcode-worker ← находит поды по лейблу
  ports:
    - port: 80 ← порт Service
      targetPort: 8080 ← порт контейнера
```

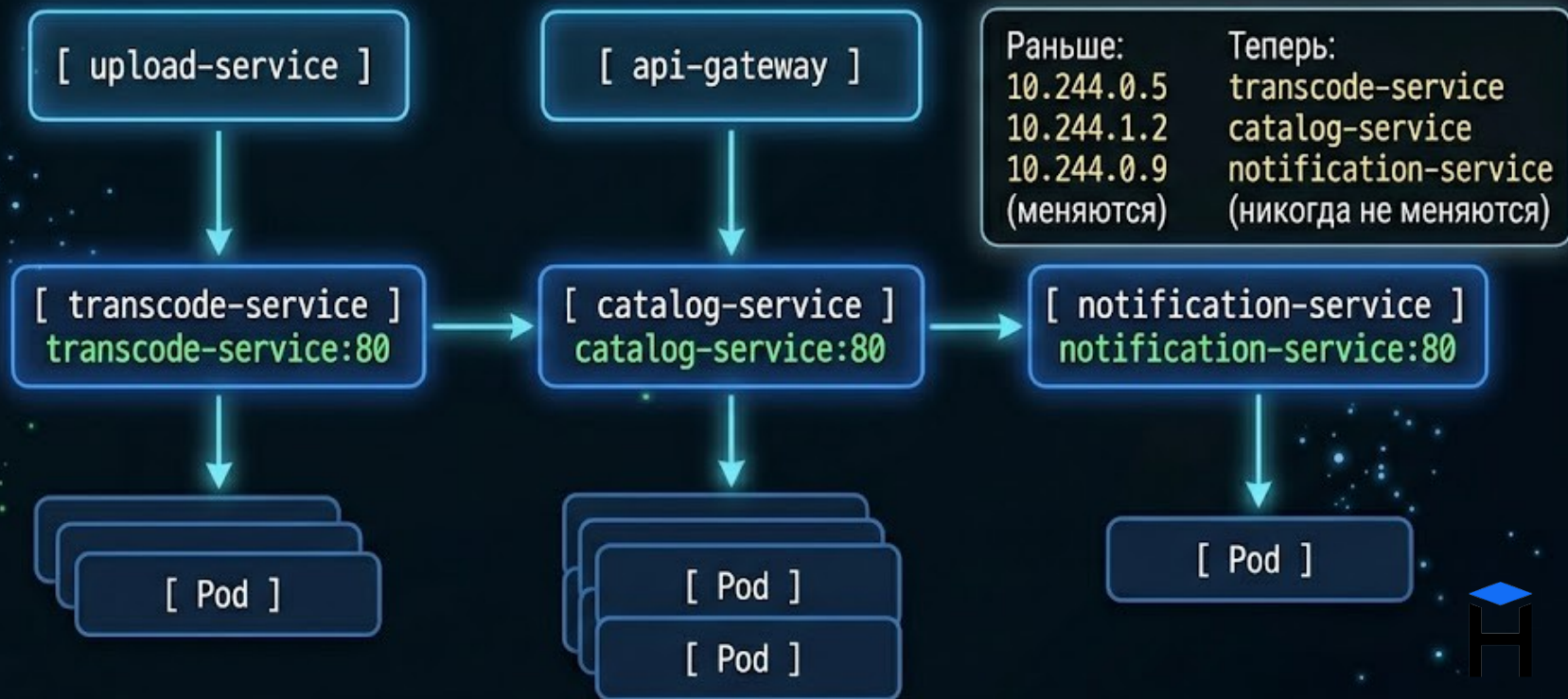
Теперь в коде:

```
TRANSCODE_URL = 'http://transcode-service:80'
```

Больше не меняется никогда.

✓ Service = стабильный DNS + балансировка нагрузки между подами. Бесплатно.

STREAMFLOW — КАЖДЫЙ СЕРВИС ПОЛУЧИЛ ИМЯ



✓ Код написан один раз. Поды перезапускаются — сервисы находят друг друга **всегда**.

Демо · Модуль 2 · Service и DNS

Применяем Service, проверяем DNS-резолвинг внутри кластера

```
# 1. Применяем Service
kubectl apply -f streamflow.yaml
kubectl get services

# 2. Смотрим что Service видит поды
kubectl describe service catalog-service
# Endpoints: должны быть IP подов

# 3. Заходим внутрь пода
kubectl exec -it <pod_name> -- sh

# 4. Внутри пода — стучим по DNS-имени
curl http://catalog-service/healthz
# Kubernetes резолвит имя → IP Service → поды

# 5. Убиваем под — IP меняется, имя работает
kubectl delete pod <catalog_pod>
curl http://catalog-service/healthz # всё ещё работает
```



DNS-имя Service остаётся неизменным при любых перезапусках подов

3

Конфигурация без боли

ConfigMap · Secret



ПРОБЛЕМА — КОНФИГ ВЕЗДЕ И НИГДЕ

Захардкожено в образе

```
# transcode-worker/config.py  
  
CATALOG_URL = "http://10.244.0.5"  
QUALITY = "1080p"  
MAX_JOBS = 8
```



Хочешь поменять quality?
Пересобирай образ.
Переливай деплой.

Разный конфиг на серверах



Два разных образа. Или
один образ с багом потому
что забыли поменять.

Секреты в git

```
# config.py ← в репозитории!  
  
REDIS_URL = "redis://prod:6379"  
AWS_KEY = "AKIA..."  
DB_PASS = "supersecret123"
```



Один git blame — и все
секреты видны всей
команде навсегда.

❌ Конфиг не должен жить в образе. Образ — это код. Конфиг — это среда.

CONFIGMAP – КОНФИГ ЖИВЁТ В КЛАСТЕРЕ

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: stream-config
data:
  TRANSCODE_QUALITY: "720p"
  ← параметр качества
  CATALOG_URL:
    "http://catalog-service"
  MAX_CONCURRENT_JOBS: "4"
  LOG_LEVEL: "info"
```

[ConfigMap]
stream-config

↓ инжектируется как
↓ переменные окружения

[Pod: transcode-worker]

```
env:
  TRANSCODE_QUALITY=720p
  CATALOG_URL=http://catalog-service
  MAX_CONCURRENT_JOBS=4
```

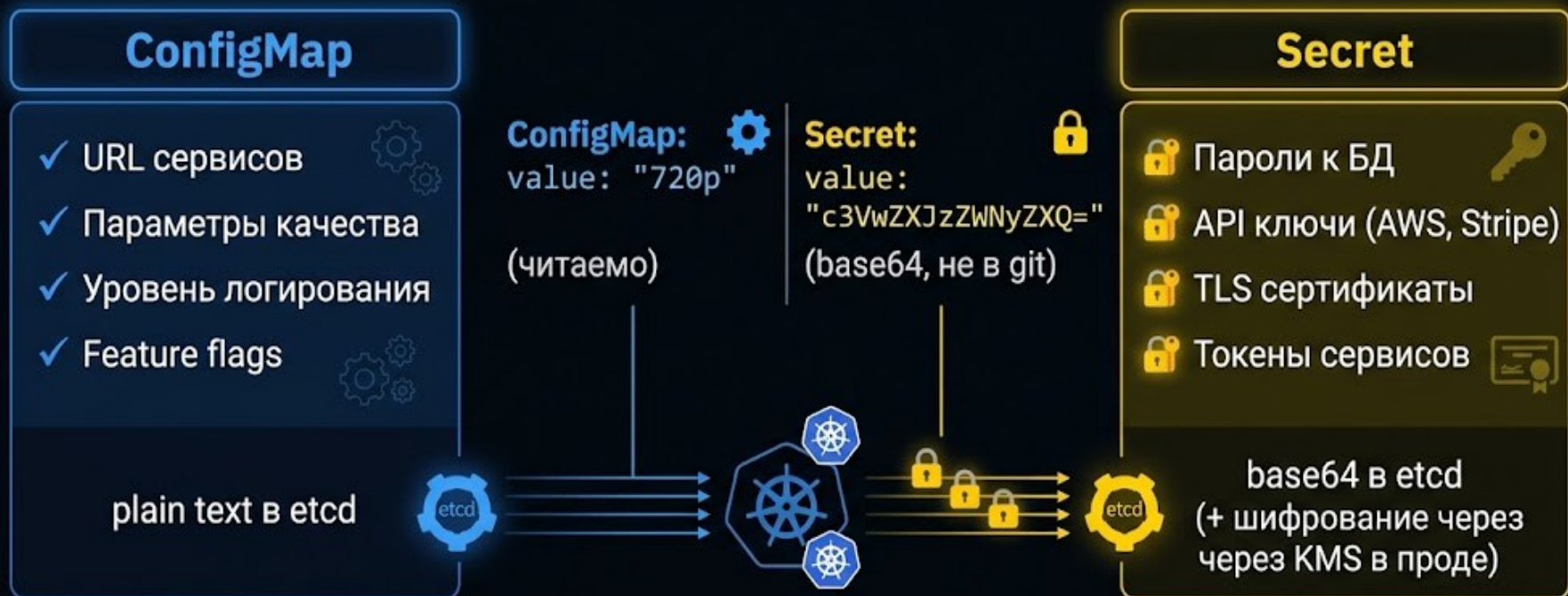
```
containers:
- name: worker
  env:
  - name: TRANSCODE_QUALITY
    valueFrom:
      configMapKeyRef:
        name: stream-config
        key: TRANSCODE_QUALITY
```

← имя ConfigMap ← ключ

Меняем ConfigMap →
перезапускаем под →
новый конфиг подхвачен.
Образ не трогаем.

✓ Один образ работает на staging и prod. Разный конфиг — разные ConfigMap.

SECRET — ТО ЖЕ ЧТО CONFIGMAP, НО ДЛЯ ПАРОЛЕЙ



! Secret — это отдельная большая тема: RBAC, external secret stores, rotation. Сегодня знаем что существует. Следующее занятие — только про это.



Демо · Модуль 3 · ConfigMap

Применяем ConfigMap, проверяем что переменная попала в под, меняем значение

```
# 1. Применяем ConfigMap и Deployment
kubectl apply -f streamflow.yaml

# 2. Проверяем что переменная попала в под
kubectl exec -it <pod_name> -- sh
env | grep WELCOME
# WELCOME_MSG=StreamFlow v1.0

# 3. Меняем значение в streamflow.yaml
# WELCOME_MSG: "StreamFlow v2.0"
kubectl apply -f streamflow.yaml

# 4. Перезапускаем поды чтобы подхватили новый конфиг
kubectl rollout restart deployment transcoder-deploy
kubectl exec -it <new_pod> -- env | grep WELCOME
# WELCOME_MSG=StreamFlow v2.0 ← новое значение
```



Образ не пересобирался — только ConfigMap обновился

4

Снаружи видно только то что нужно

Ingress



ПРОБЛЕМА — ВСЁ ТОРЧИТ НАРУЖУ



Что может пойти не так:

- Прямой запрос к Redis
→ данные утекли
- Прямой запрос к catalog
→ обход api-gateway
→ обход авторизации
- DDoS на transcode-worker
→ очередь встала



[Redis :6379]

185.12.44.10:6379

← доступен
из интернета



[catalog-service]

185.12.44.11:8081

← доступен
из интернета



[transcode-worker]

185.12.44.12:8082

← доступен
из интернета



[api-gateway]

185.12.44.13:8081

← доступен
из интернета



[upload-service]

185.12.44.11:8081

← доступен
из интернета

Реальный кейс:



MongoDB без пароля

+ доступна из интернета
= 27017 портов в shodan.io



Shodan



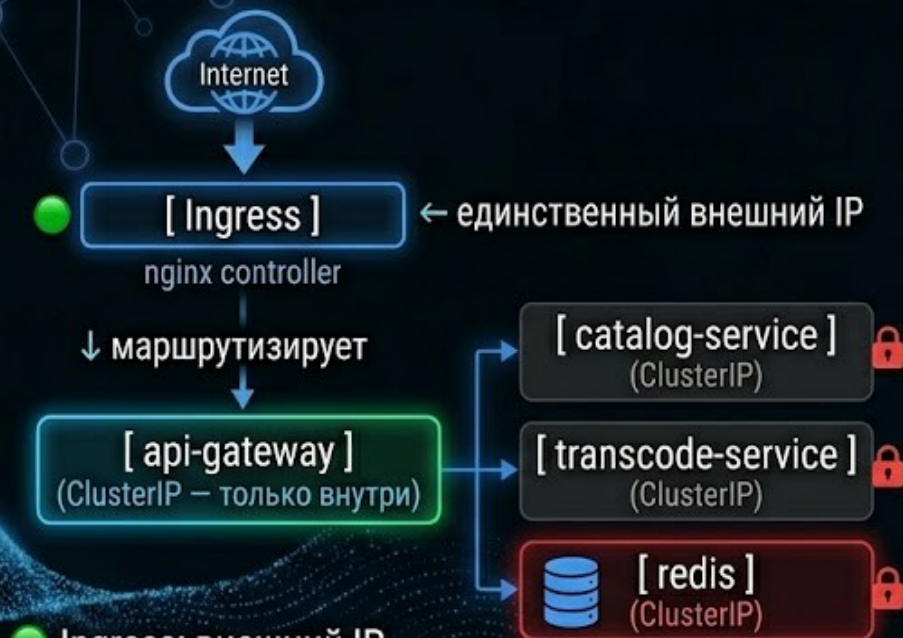
“Ваш Redis может быть
в этом списке
прямо сейчас”



✗ Наружу должен смотреть только один сервис.
Всё остальное — только внутри кластера.



INGRESS – ОДИН ВХОД, ВСЁ ОСТАЛЬНОЕ ЗАКРЫТО



● Ingress: внешний IP

🔒 Всё остальное: ClusterIP, снаружи недоступно

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: stream-ingress    ← перезаписываем путь
  annotations:
    nginx.ingress.kubernetes.io/
    rewrite-target: /      ← rewrite target annotated
spec:
  ingressClassName: nginx ← используем nginx
  rules:
    - http:
        paths:
          - path: /         ← перезаписываем путь
            pathType: Prefix
            backend:
              service:
                name: api-api-gateway ← все запросы
                port: 80              ← идут сюда
```

✓ Один внешний IP. Один сервис снаружи.
Redis, catalog, transcoder — только изнутри кластера.



STREAMFLOW — СЕТЬ ПОСЛЕ INGRESS



Раньше:

- 5 внешних IP
- Redis открыт
- обход авторизации возможен

Теперь:

- 1 внешний IP (Ingress)
- Redis **закрит**
- обход авторизации невозможен

✓ Пользователь видит только Ingress.
Вся внутренняя архитектура скрыта.



Демо · Модуль 4 · Ingress

Включаем Ingress в minikube, проверяем доступность снаружи

```
# 1. Включаем Ingress аддон (один раз)
minikube addons enable ingress

# 2. Применяем манифест
kubectl apply -f streamflow.yaml

# 3. Смотрим что Ingress создан
kubectl get ingress

# 4. Запускаем туннель (держать открытым!)
minikube tunnel

# 5. В другом терминале — проверяем
curl http://localhost/
# Должен ответить api-gateway

# 6. catalog-service закрыт снаружи
# только через api-gateway → catalog-service
```



minikube tunnel нужно держать открытым во время теста

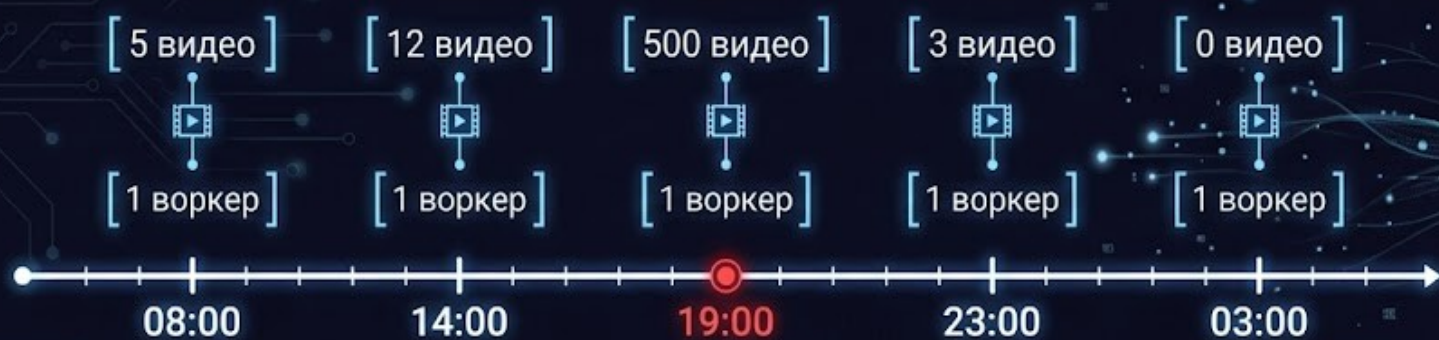
5

Пиковая нагрузка

Масштабирование · НРА



ПРОБЛЕМА — ОДИН ВОРКЕР НЕ СПРАВЛЯЕТСЯ



В 19:00:

- 500 видео в очереди
- Один воркер берёт по одному
- Пользователи ждут часами
- Жалобы в поддержку

[1 воркер]

очередь растёт
задержка: часы

[1 воркер]

простаивает
жжёт деньги

В 03:00:

- Очередь пуста
- Воркер простаивает
- CPU: 0%
- Деньги идут
- Никакой пользы

✗ Один воркер — всегда **компромисс**: либо не справляется с пиком, либо простаивает в тишине.

МАСШТАБИРОВАНИЕ — БОЛЬШЕ ВОРКЕРОВ КОГДА НУЖНО

(Сегодня на демо)

Ручное масштабирование

```
# пик нагрузки — добавляем воркеры
kubectl scale deployment transcode-worker \
  --replicas=10

# нагрузка спала — убираем
kubectl scale deployment transcode-worker \
  --replicas=1
```



Работает. Но кто-то должен следить и делать это руками.

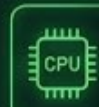
(В реальном проде)

HPA — автоматически

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    name: transcode-worker
  minReplicas: 1
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        averageUtilization: 70
```



← минимум ночью
← максимум в пик



HPA
CPU > 70% → добавляет
поды
CPU < 70% → HPA убирает
поды



Kubernetes смотрит на метрики и сам решает сколько подов нужно.

✓ **Ручное масштабирование** — понять механику. **HPA** — следующий шаг когда освоите базу



STREAMFLOW – МАСШТАБИРОВАНИЕ В ДЕЙСТВИИ



Раньше (1 воркер):
19:00 – очередь растёт
задержка: часы
03:00 – простой
Стоимость: фиксированная

Теперь (1-20 подов):
19:00 – 10 подов, очередь тает
задержка: минуты
03:00 – 1 под, минимум затрат
Стоимость: пропорциональна нагрузке

✓ Платим за то что используем. Пользователи не ждут. Воркеры не простаивают.

Демо · Модуль 5 · Масштабирование

Масштабируем transcode-worker вручную, наблюдаем как поды появляются и исчезают

```
# 1. Смотрим текущее состояние
kubectl get pods
kubectl get deployment transcoder-deploy

# 2. Масштабируем вверх — пик нагрузки
kubectl scale deployment transcoder-deploy --replicas=5
kubectl get pods -w
# Наблюдаем: 4 новых пода появляются

# 3. Смотрим распределение по нодам
kubectl get pods -o wide

# 4. Масштабируем вниз — нагрузка спала
kubectl scale deployment transcoder-deploy --replicas=1
kubectl get pods -w
```



HPA (Horizontal Pod Autoscaler) делает это автоматически по метрикам CPU

6

Деплой без страха

RollingUpdate · Rollback



ПРОБЛЕМА – ДЕПЛОЙ КАК РУССКАЯ РУЛЕТКА

Что пошло не так:

- Убили старый под до проверки нового
- Новый под упал при старте
- Нет старой версии – некуда откатиться быстро
- 45 минут даунтайма 



Типичная реакция:

- 15:03 – "всё упало"
- 15:05 – паника в Slack 
- 15:10 – будим DevOps 
- 15:20 – SSH на сервер 
- 15:35 – git stash, docker up 
- 15:45 – "кажется поднялось"
- 16:30 – постмортем 



Деплой в пятницу вечером — источник корпоративных легенд.
Не потому что разработчики плохие. Потому что процесс опасный.





ROLLINGUPDATE — ДЕПЛОЙ БЕЗ ДАУНТАЙМА



Шаг 1: Начало деплоя v2 

[Pod v1] [Pod v1] [Pod v1] ← все живые, трафик идёт

Шаг 2: Поднимаем новый под 

[Pod v1] [Pod v1] [Pod v2] ← liveness probe проверяет v2

↑ ждём: probe OK?

Шаг 3: Probe прошла — убиваем старый 

[Pod v1] [Pod v2] [Pod v2] ← трафик идёт всё время

Шаг 4: Повторяем до конца 

[Pod v2] [Pod v2] [Pod v2] ← деплой завершён,
даунтайм: 0



Пользователи не заметили ничего



Что если v2 сломан? 

[Pod v1] [Pod v2 ] [Pod v2 ]



```
kubectl rollout undo deployment stream-service
```



[Pod v1] [Pod v1] [Pod v1]

10 секунд. Без SSH. Без паники. 

```
# посмотреть историю
kubectl rollout history deployment stream-service
# откатиться на предыдущую версию
kubectl rollout undo deployment stream-service
# откатиться на конкретную версию
kubectl rollout undo deployment stream-service \
--to-revision=2
```



Kubernetes поднимает новый под, проверяет что живой, **только потом** убивает старый. Что-то пошло не так — одна команда назад.



Демо · Модуль 6 · RollingUpdate

Деплоим новую версию, наблюдаем rolling update, делаем rollback

```
# 1. Меняем тег образа в streamflow.yaml
# image: streamflow-mock:v2
kubectl apply -f streamflow.yaml

# 2. Наблюдаем rolling update
kubectl rollout status deployment catalog-deploy
kubectl get pods -w

# 3. Симулируем плохой деплой
# image: streamflow-mock:v999
kubectl apply -f streamflow.yaml
kubectl get pods # ImagePullBackOff на новых, старые живы!

# 4. Откатываемся одной командой
kubectl rollout undo deployment stream-service
kubectl get pods -w # старая версия вернулась
kubectl rollout history deployment catalog-deploy
```





STREAMFLOW — ДЕПЛОЙ БЕЗ СТРАХА



Деплой stream-service v2:

Раньше



```
15:00 ssh user@185.12.44.10
15:01 docker stop stream-service ← даунтайм начался 
15:02 docker pull streamflow:v2
15:03 docker run streamflow:v2 ← молимся  
15:04 [упало] CrashLoopBackOff ← даунтайм продолжается
15:45 git stash, docker run v1 ← 45 минут даунтайма 
```

Деплой stream-service v2:

Теперь

```
# меняем тег в манифесте
image: streamflow:v2

kubectl apply -f deployment.yaml ← RollingUpdate начался 
[Pod v1] [Pod v1] [Pod v1] ← старые поды живы 
                                ← новые проверяются probe
                                ← даунтайм: 0 секунд 

# если сломалось:
kubectl rollout undo deployment ← 10 секунд, всё вернулось
```

Checklist — Что мы построили сегодня

- ✓ Pod + Deployment — сервис не падает навсегда
- ✓ Liveness Probe — зависший сервис перезапускается сам
- ✓ Service — сервисы находят друг друга по имени
- ✓ ConfigMap — конфиг отдельно от кода
- ✓ ConfigMap — конфиг отдельно от кода
- ✓ Ingress — один вход, всё остальное закрыто
- ✓ Масштабирование — воркеров столько сколько нужно
- ✓ RollingUpdate — деплой без даунтайма и страха

✓ Можно деплоить **в пятницу вечером**. Kubernetes поймает тебя если что-то пойдёт не так.



Что мы построили сегодня



Pod + Deployment

сервис не падает навсегда



Liveness Probe

зависший сервис перезапускается сам



Service

сервисы находят друг друга по имени



ConfigMap

конфиг отдельно от кода



Ingress

один вход, всё остальное закрыто



Масштабирование

воркеров столько сколько нужно



RollingUpdate

деплой без даунтайма и страха



Можно деплоить в пятницу вечером.

Шпаргалка — kubectl

```
kubectl get all
```

Показать всё: поды, сервисы, деплойменты

```
kubectl get pods -w
```

Следить за подами в реальном времени

```
kubectl describe pod <n>
```

Найти причину почему под не стартует (Events)

```
kubectl logs <n>
```

Логи пода

```
kubectl logs <n> --previous
```

Логи до последнего рестарта

```
kubectl logs -f -l app=catalog
```

Следить за логами всех подов catalog

```
kubectl exec -it <n> -- sh
```

Зайти внутрь пода

```
kubectl apply -f file.yaml
```

Применить манифест

```
kubectl scale deployment <n> --replicas=N
```

Изменить количество реплик

```
kubectl rollout undo deployment <n>
```

Откатить деплой

```
minikube tunnel
```

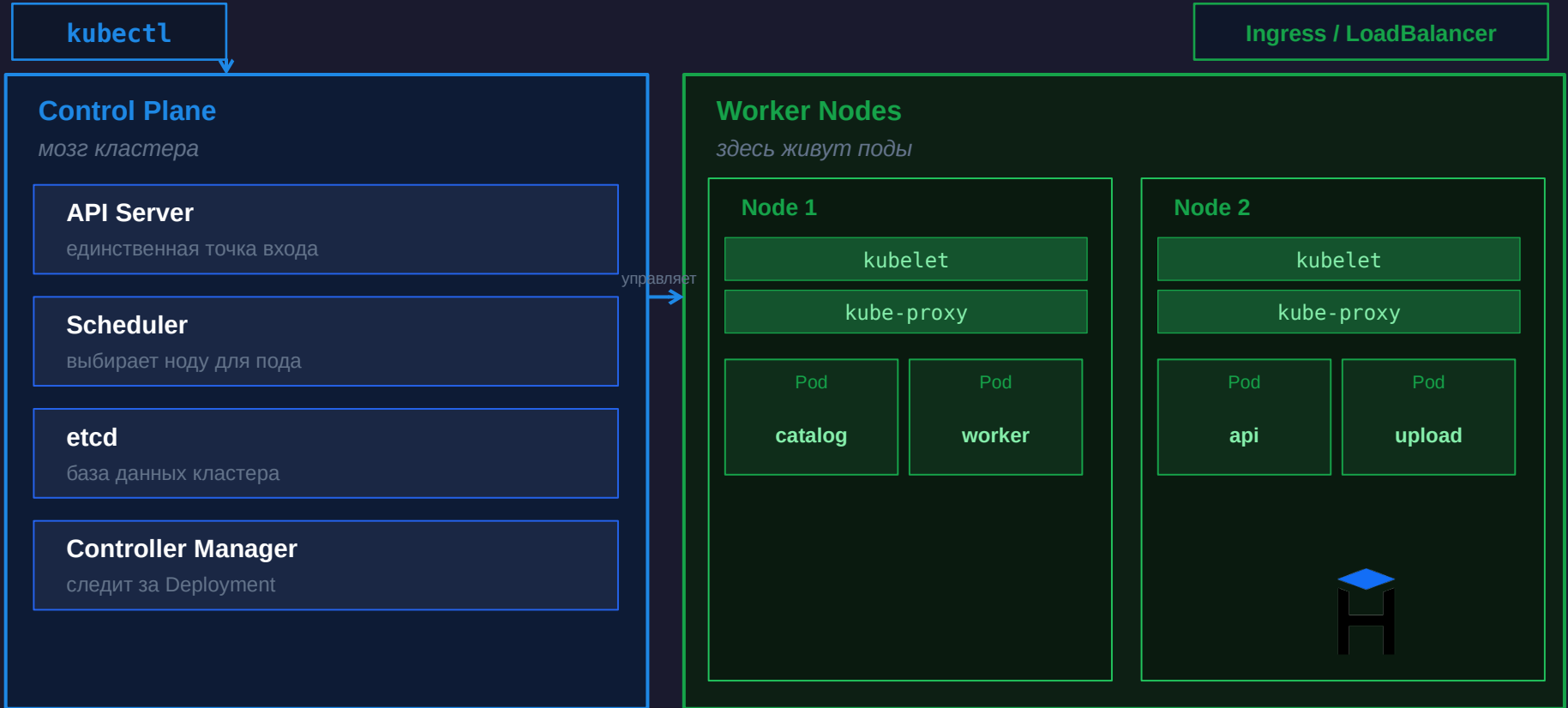
Пробросить Ingress на localhost

```
minikube dashboard
```

Открыть UI в браузере



Как устроен Kubernetes кластер



Вы уже работали со всеми этими компонентами — просто не знали их названий.

Helm — пакетный менеджер для Kubernetes

Проблема

configmap.yaml

service-api.yaml

deployment-api.yaml

service-catalog.yaml

deployment-catalog.yaml

service-transcoder.yaml

deployment-transcoder.yaml

ingress.yaml



8 файлов. Для каждого окружения — staging, prod — отдельный набор.

Поддерживать это вручную — боль.

Helm Chart

streamflow/

← папка чарта

Chart.yaml

← имя, версия, описание

values.yaml

← все настройки в одном месте

templates/

← шаблоны ваших YAML файлов

deployment.yaml

← с переменными {{ .Values.* }}

service.yaml

ingress.yaml

Одна команда вместо восьми kubectl apply:

```
helm install streamflow ./streamflow
```

```
helm upgrade streamflow ./streamflow
```

```
helm rollback streamflow 1
```



Helm — следующая тема. Сегодня знаем что существует и зачем нужен.

Спасибо за внимание

Заполняйте анкету после активности!
Вам несложно,
Хекслету - полезно.

